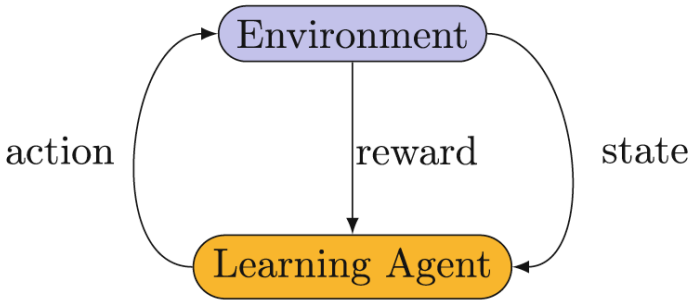
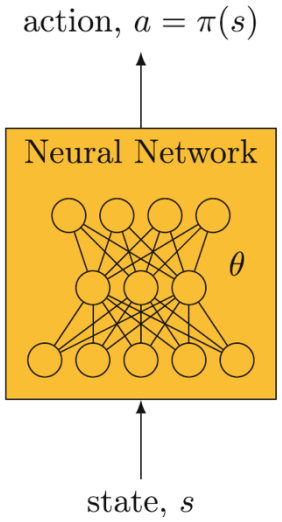
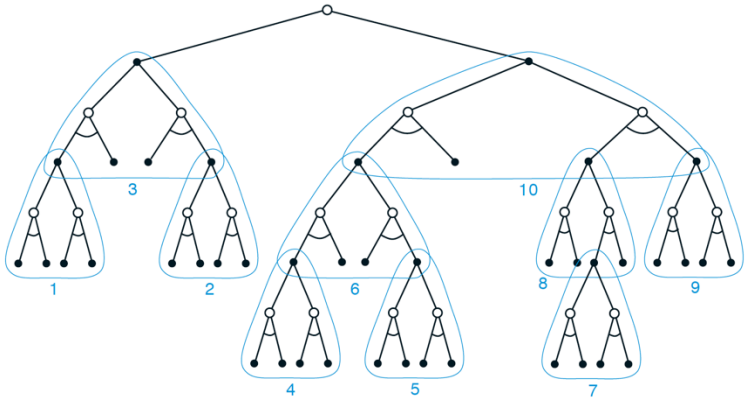
Markov Decision Processes

Forest Agostinelli
University of South Carolina

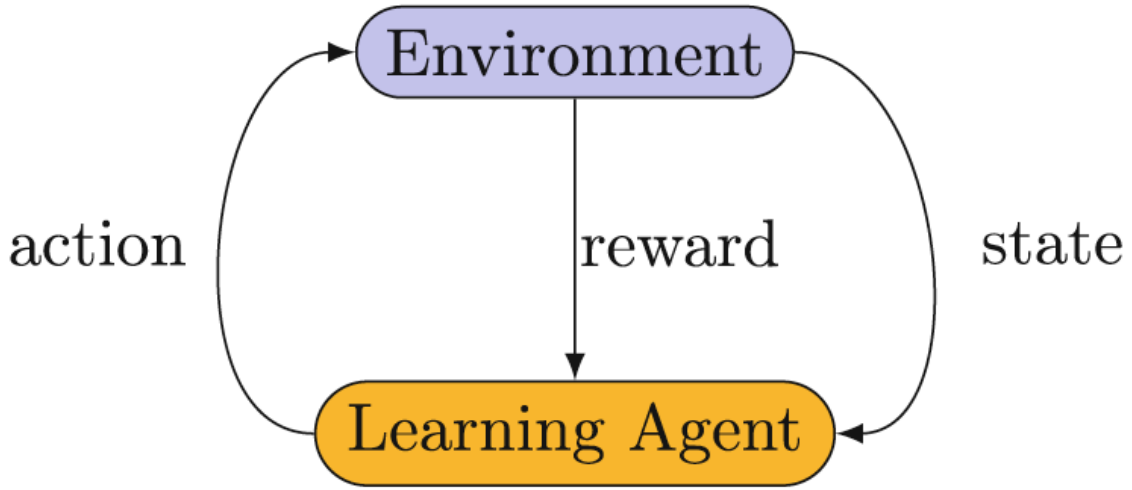






Review

- **Reinforcement learning**: learning to map states to actions so that we maximize the reward we receive from the environment.
- This mapping of states to actions is called a policy function.
  - Deterministic: $a = \pi(s)$
  - Stochastic: $\pi(a|s) = P(A = a|S = s)$
- At each time step t
  - In state S_t, agent takes action A_t
  - Based on state S_t and action A_t, the environment transitions to state S_{t+1} and outputs reward R_{t+1}



Notation

- For notation, capital letters (i.e. S_t) are used for random variables and lowercase letters (i.e. s) are used for a particular value of that random variable.
- For states, s typically refers to the current state and s' typically refers to the next state

Outline

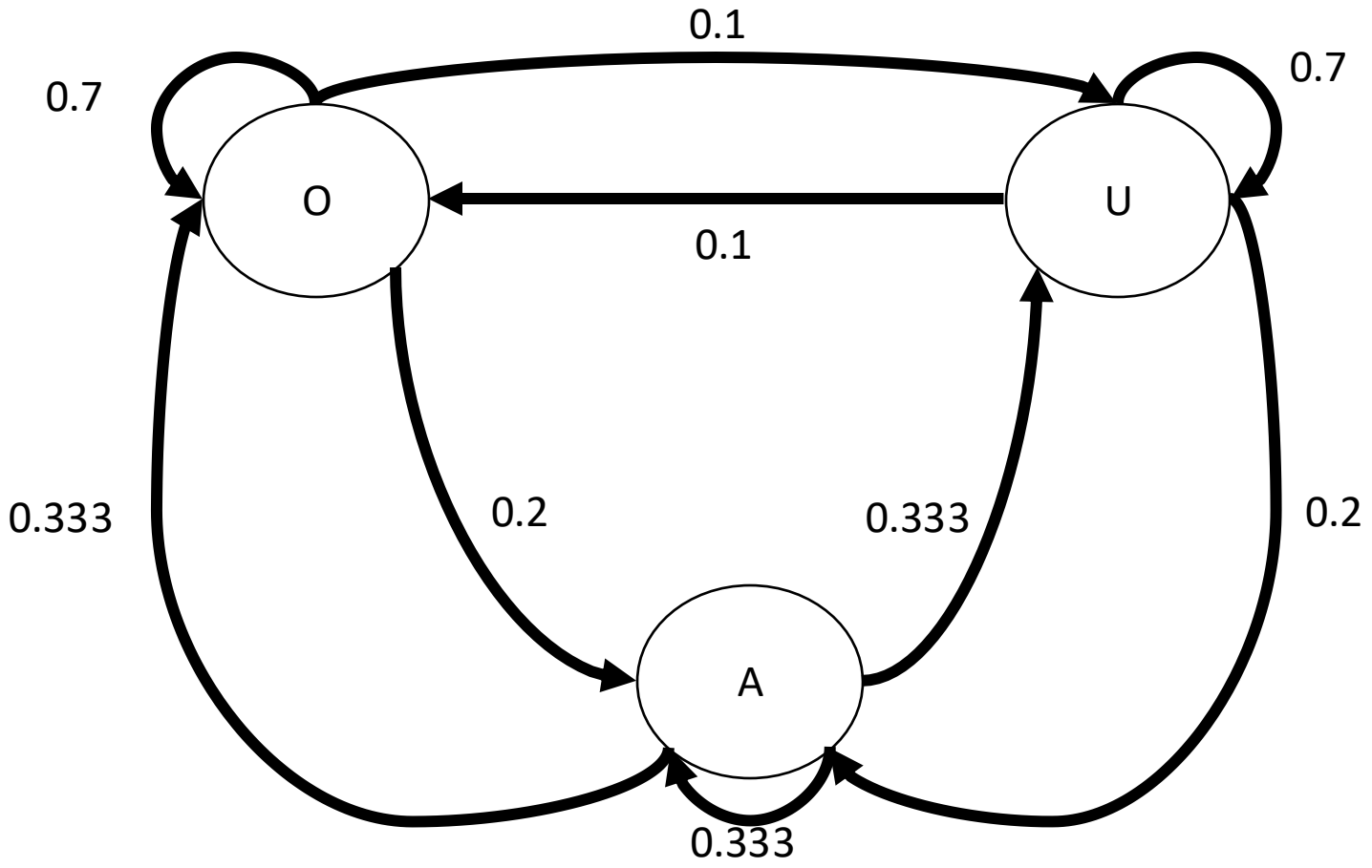
- Markov Processes
- Markov Reward Processes
  - Solving MRPs
- Markov Decision Processes
  - Solving MDPs

Markov Process

- Models a sequence of random variable S_t, often referred to as the state, whose future value only depends on the current value
- $P(S_{t+1} = s' | S_t = s) = P(S_{t+1} = s' | S_t = s, S_{t-1} = s_{t-1}, \dots, S_0 = s_0)$
  - Referred to as the dynamics or transition probabilities
- In other words, the future is independent of the past states given the present state
- This is also referred to as the Markov property
- In the discrete time setting, which is the setting we use for reinforcement learning, this is also referred to as a Markov chain

Markov Process: Hospital Example

- The hospital can be in three states
  - Over capacity (O): Insufficient staff and resources
  - Under capacity (U): More than enough staff and resources
  - At capacity (A): About the right number of staff and resources for patients
- With Markov processes, we are often interested in the how the state changes over time
  - I.e. Does a steady state distribution exist, if so, what is it?



Outline

- Markov Processes
- Markov Reward Processes
  - Solving MRPs
- Markov Decision Processes
  - Solving MDPs

Markov Reward Process (MRP)

- There may be some scalar value we can assign to the transitions between states to indicate how good each transition is
- Dynamics: $P(S_{t+1} = s', R_{t+1} = r | S_t = s)$
- The state transition dynamics can be computed as
  - $p(s'|s) = \sum_{r \in \mathcal{R}} p(s', r|s)$
- The reward dynamics of a state can be computed as
  - $r(s) = \sum_{r \in \mathcal{R}} r \sum_{s' \in \mathcal{S}} p(s', r|s) = \mathbb{E}[R_{t+1}|S_t = s]$

MRP: Return and Value

- We can calculate the return, which is the sum of rewards after timestep t
  - $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} \ldots$
- We can then model the expected value of a state using a state-value function
  - $v(s) = \mathbb{E}[G_t | S_t = s] = \mathbb{E}[\sum_{k=0}^{\infty} \gamma^k R_{t+1+k} \, | S_t = s]$
- We use a discount factor γ to discount future rewards
  - $0 \leq \gamma \leq 1$
  - Encodes how far in the future we want to look
    - $\gamma = 0$ means we only care about the immediate reward
    - $\gamma = 1$ means we care about all future rewards equally
  - Can be mathematically convenient as return will be finite if $\gamma < 1$ and the rewards are bounded (geometric series)

Outline

- Markov Processes
- Markov Reward Processes
  - Solving MRPs
- Markov Decision Processes
  - Solving MDPs

Solving MRPs

- $v(s) = \mathbb{E}[G_t | S_t = s] = \mathbb{E}[\sum_{k=0}^{\infty} \gamma^k R_{t+1+k} \, | S_t = s]$
- We can derive that
  - $v(s) = r(s) + \gamma \sum_{s'} p(s'|s) v(s')$
- We can then use this to find the value for a given state using linear algebra
  - V and R are vectors of length $|\mathcal{S}|$ where $\mathcal{S}$ is the state space
  - P is a $|S|$x$|S|$ matrix where each index ij indicates the probability of transitioning from state i to state j
- $V = R + \gamma P V$
- $V - \gamma P V = R$
- $V(I - \gamma P) = R$

- Concept check, when might we have a problem when trying to invert?
  - $(I - \gamma P)^{-1}$

MRP: Defining Value in Terms of Itself

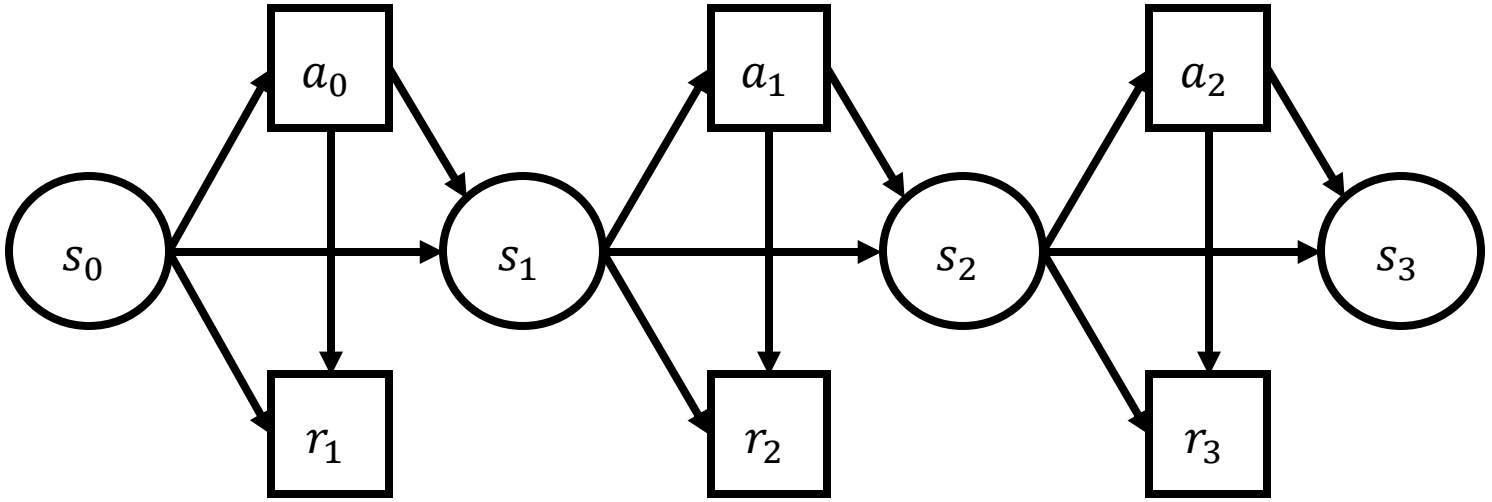
- $v(s) = \mathbb{E}[G_t | S_t = s] = \mathbb{E}[\sum_{k=0}^{\infty} \gamma^k R_{t+1+k} | S_t = s]$
- How can we show
  - $v(s) = r(s) + \gamma \sum_{s'} p(s'|s) v(s')$
- $v(s) = \mathbb{E}[R_{t+1} + \gamma G_{t+1} | S_t = s] = \mathbb{E}[R_{t+1} | S_t = s] + \gamma \mathbb{E}[G_{t+1} | S_t = s]$
- $\mathbb{E}[G_{t+1} | S_t = s] = \sum_{g'} p(g'|s)\, g'$
- $\mathbb{E}[G_{t+1} | S_t = s] = \sum_{g'} \sum_{s'} p(g', s'|s)\, g'$
- $\mathbb{E}[G_{t+1} | S_t = s] = \sum_{g'} \sum_{s'} p(g'|s', s)\, p(s'|s) g'$
- $\mathbb{E}[G_{t+1} | S_t = s] = \sum_{g'} \sum_{s'} p(g'|s')\, p(s'|s) g'$
- $\mathbb{E}[G_{t+1} | S_t = s] = \sum_{s'} p(s'|s) \sum_{g'} p(g'|s')\, g'$
- $\mathbb{E}[G_{t+1} | S_t = s] = \sum_{s'} p(s'|s) \mathbb{E}[G_{t+1} | S_{t+1} = s']$
- $v(s) = \mathbb{E}[R_{t+1} | S_t = s] + \gamma \sum_{s'} p(s'|s) \mathbb{E}[G_{t+1} | S_{t+1} = s']$
- $v(s) = r(s) + \gamma \sum_{s'} p(s'|s) v(s')$

Outline

- Markov Processes
- Markov Reward Processes
  - Solving MRPs
- Markov Decision Processes
  - Solving MDPs

Markov Decision Processes (MDPs)

- **States**
- **Actions**
- **Transition Probabilities:** $P(S_{t+1} = s', R_{t+1} = r | S_t = s, A_t = a)$
  - Defines the dynamics of the MDP



Markov Decision Processes (MDPs)

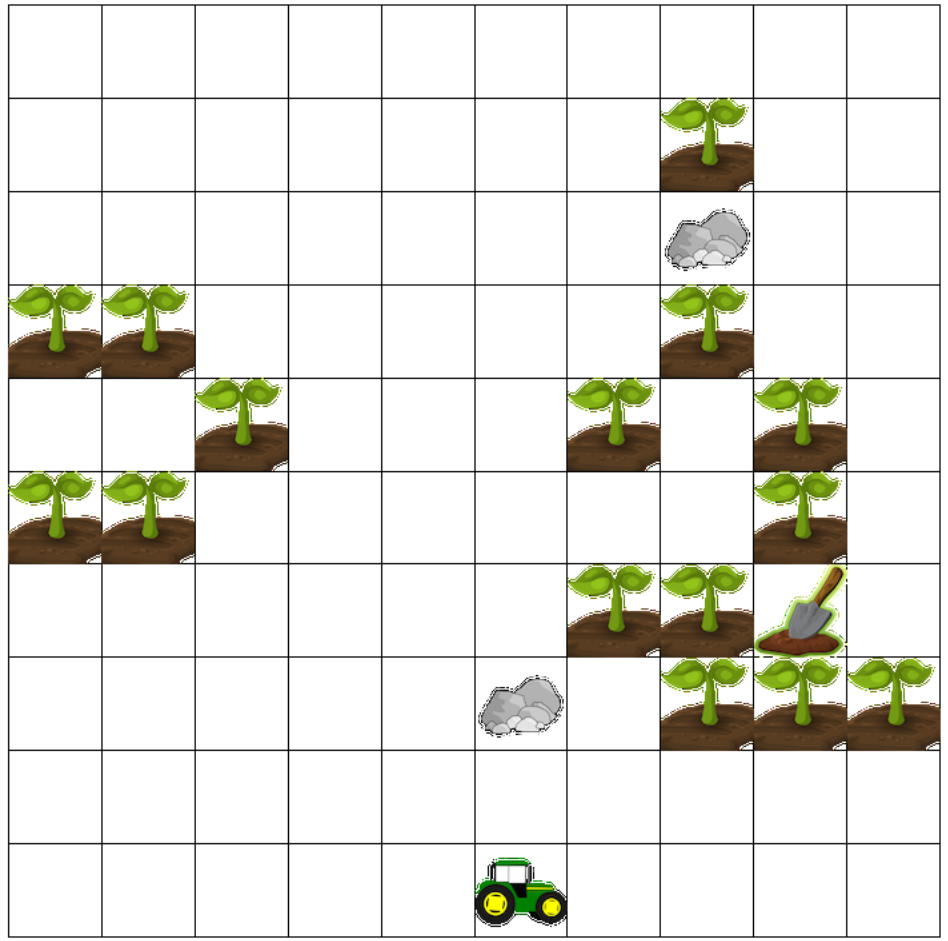
- The state-transition probabilities can be obtained from the transition probabilities
  - $p(s'|s,a) = \sum_{r \in \mathcal{R}} p(s', r|s, a)$
- The expected reward can be obtained from the transition probabilities
  - $r(s,a) = \sum_{r \in \mathcal{R}} r \sum_{s' \in \mathcal{S}} p(s', r|s, a) = \mathbb{E}[R_{t+1}|S_t = s, A_t = a]$
- For now, we assume the state and actions are discrete and finite, however, this restriction can be relaxed to be continuous and infinite

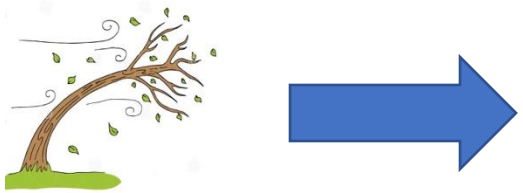
The Markov Property for MDPs

- Our policy at timepoint t is only based on the current state s
  - $\pi(a|s) = P(A_t = a|S_t = s)$
- However, the agent has a history up until S_t
  - $\mathrm{H_t} = S_0, A_0, R_1 S_1, A_1, R_2 \ldots S_{t-1}, A_{t-1}, R_t, S_t$
- We assume that all relevant information about the future is contained in the current state and action
  - $P(S_{t+1} = s', R_{t+1} = r|S_t = s, A_t = a) = P(S_{t+1} = s', R_{t+1} = r|H_t = h_{t+1}, A_t = a)$

MDPs: AI Farm

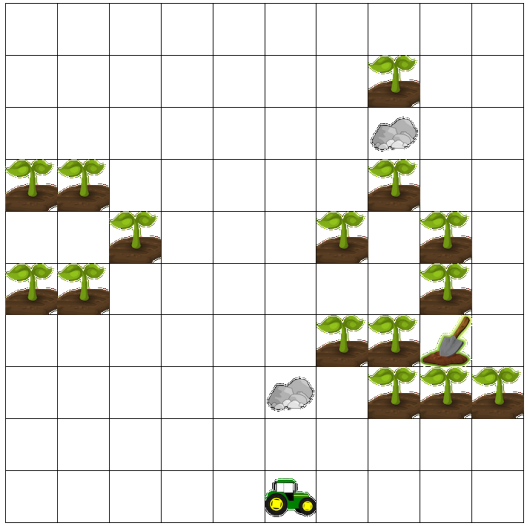
- State: Location of agent, plants, rocks, and goal
- Actions: up, down, left, right
- Rewards
  - Driving on a plant: -50
  - Driving on a rock: -10
  - Driving on any other space: -1
- $P(S_{t+1} = s', R_{t+1} = r | S_t = s, A_t = a)$
- Strong winds?



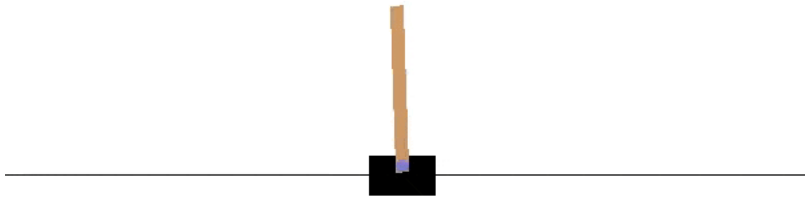


Breakout Session: State Representations

- **AI Farm - Harvesting**
- Actions: Up, down left, right, harvest
- Rewards: Same as before and +1 for every crop harvested. Can only harvest crop once.
- Episode ends when all crops are harvested
- How many states?



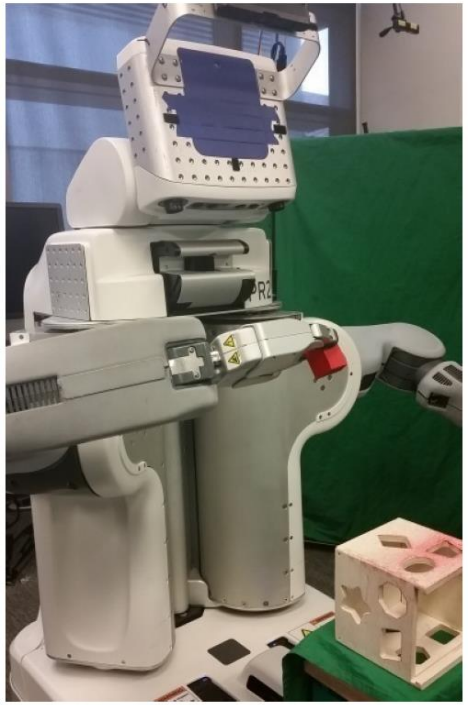
- **Cartpole**
- Actions: apply force left, apply force right
- Rewards: +1 for every step
- Episode ends when the pole falls over

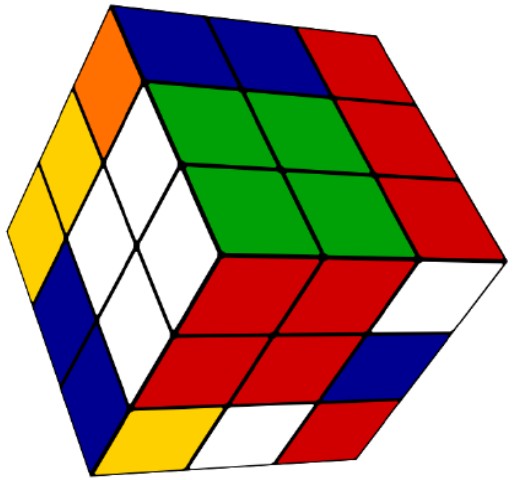


- What should the state representations be?
- Remember, the Markov property must hold: $P(S_{t+1} = s', R_{t+1} = r | S_t = s, A_t = a)$

Defining an MDP

states, actions, rewards, transition probabilities?



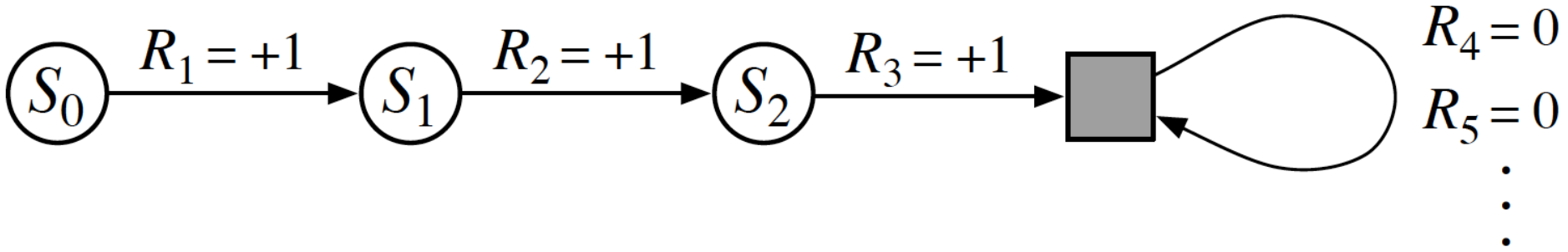


MDPs: Continuing Tasks

- There are environments in which an agent's experience is not guaranteed to terminate
- Maximizing $G_t = R_{t+1} + R_{t+2} + R_{t+3} \ldots + R_T$ becomes problematic
  - $T = \infty$
  - Therefore, it is possible that G_t could be ∞.
- Therefore, we discount the reward
  - $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} \ldots$
  - $0 \leq \gamma < 1$
    - Will converge to a finite number as long as rewards are finite
    - Geometric series: $\sum_{k=0}^{\infty} ar^k = \frac{a}{1-r}$ for $|r| < 1$
  - γ can be set to 1 if T is finite
    - In finite cases, sometimes it is still better to make it less than 1 to reduce variance when doing function approximation

MDPs: Unifying Continuing and Episodic Tasks

- Episodic tasks can be posed as continuing tasks with a terminal state that:
  - Only transitions to itself
  - Only generates rewards of 0
- The return
  - $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} \dots$
  - $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+1+k}$
  - Works both when T is finite and infinite
  - If T is not infinite, then γ can be 1

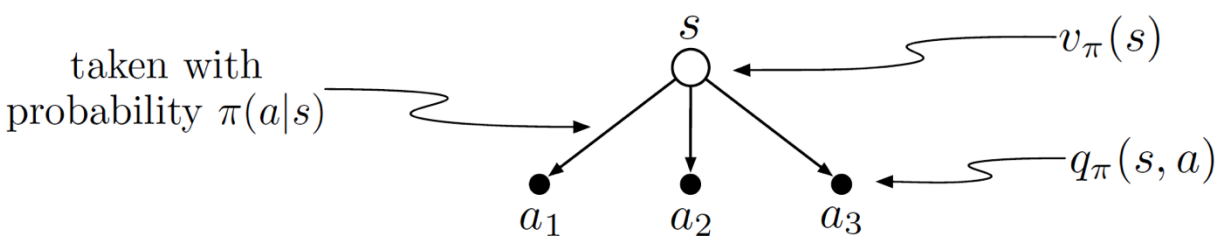


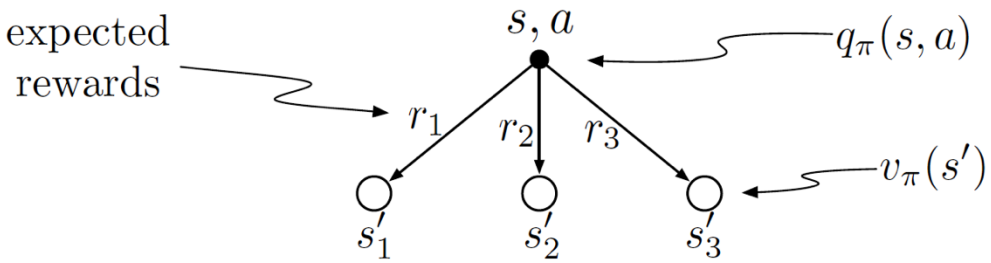
MDPs: Returns and Value

- **Episode:** Starts at some start state at timepoint 0 and ends at a special state, called the terminal state, at timepoint T
- **Return:** the sum of rewards after timestep t
  - $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} \dots$
  - We seek to maximize the expected return
- **State-value function**
  - The expected return when in state s and following policy π
  - $v_\pi(s) = \mathbb{E}_\pi[G_t | S_t = s] = \mathbb{E}_\pi[\sum_{k=0}^{\infty} \gamma^k R_{t+1+k} \, | S_t = s]$
- **Action-value function**
  - The expected return when taking action a in state s and then following policy π
  - $q_\pi(s, a) = \mathbb{E}_\pi[G_t | S_t = s, A_t = a] = \mathbb{E}_\pi[\sum_{k=0}^{\infty} \gamma^k R_{t+1+k} \, | S_t = s, A_t = a]$

Relationship Between Value Functions

- $v_\pi(s) = \sum_a \pi(a|s) q_\pi(s,a)$
- $q_\pi(s,a) = r(s,a) + \gamma \sum_{s'} p(s'|s,a)\, v_\pi(s')$

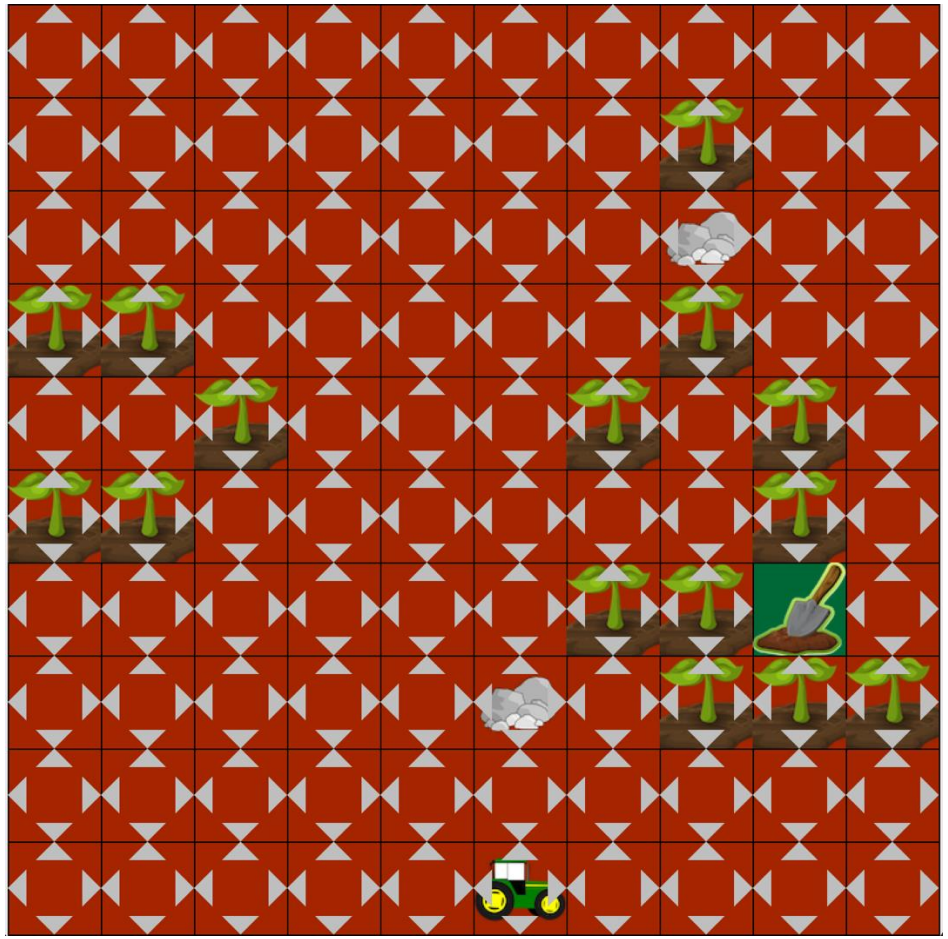




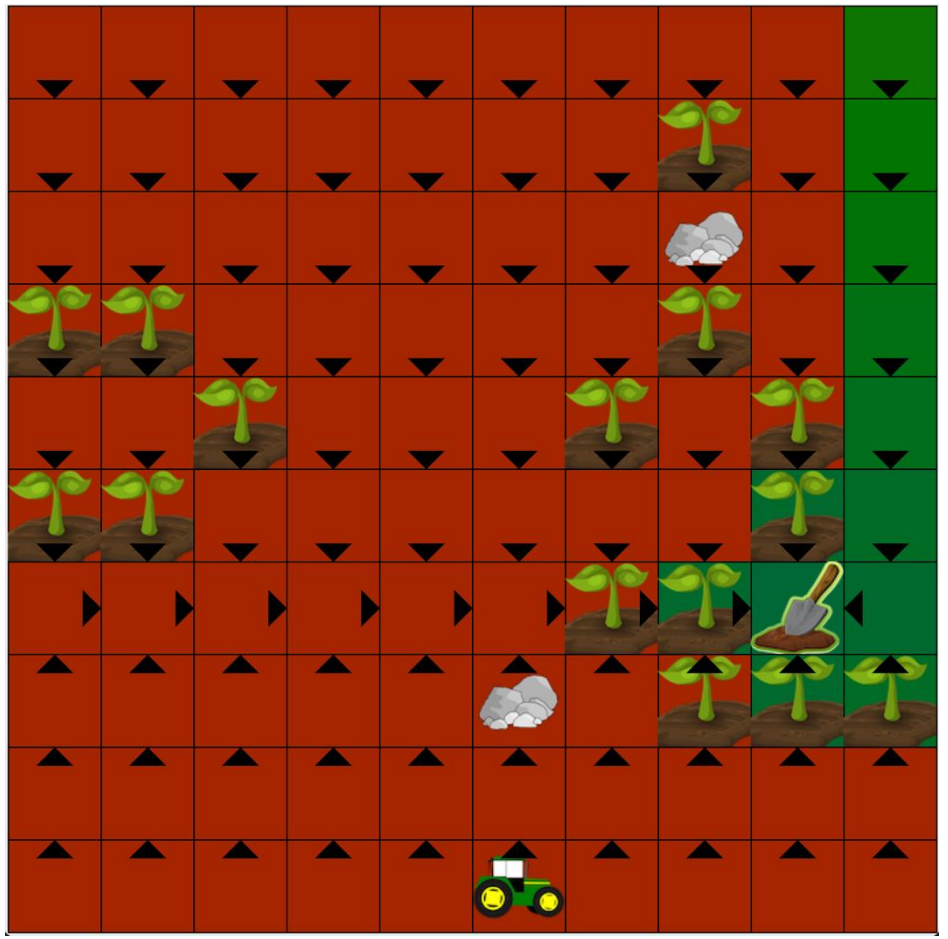
Optimal Policy and Value Function

- Which policy is better?
  - $\pi \geq \pi'$ if and only if $v_\pi(s) \geq v_{\pi'}(s)$ for all $s \in S$
- A policy that achieves the greatest possible return from any state is an optimal policy
  - $\pi_* \geq \pi'$ for all π'
- The optimal value function is the value function obtained when following the optimal policy
  - $v_*(s) = \max_\pi v_\pi(s)$
  - $q_*(s,a) = \max_\pi q_\pi(s,a)$
- Optimal policies can be obtained by behaving greedily with respect to the optimal value function
- Many RL methods first learn a value function and then induce a policy by behaving greedily with respect to the value function
- Is the optimal policy unique?
- Is the optimal value function unique?

Value Function Visualizations

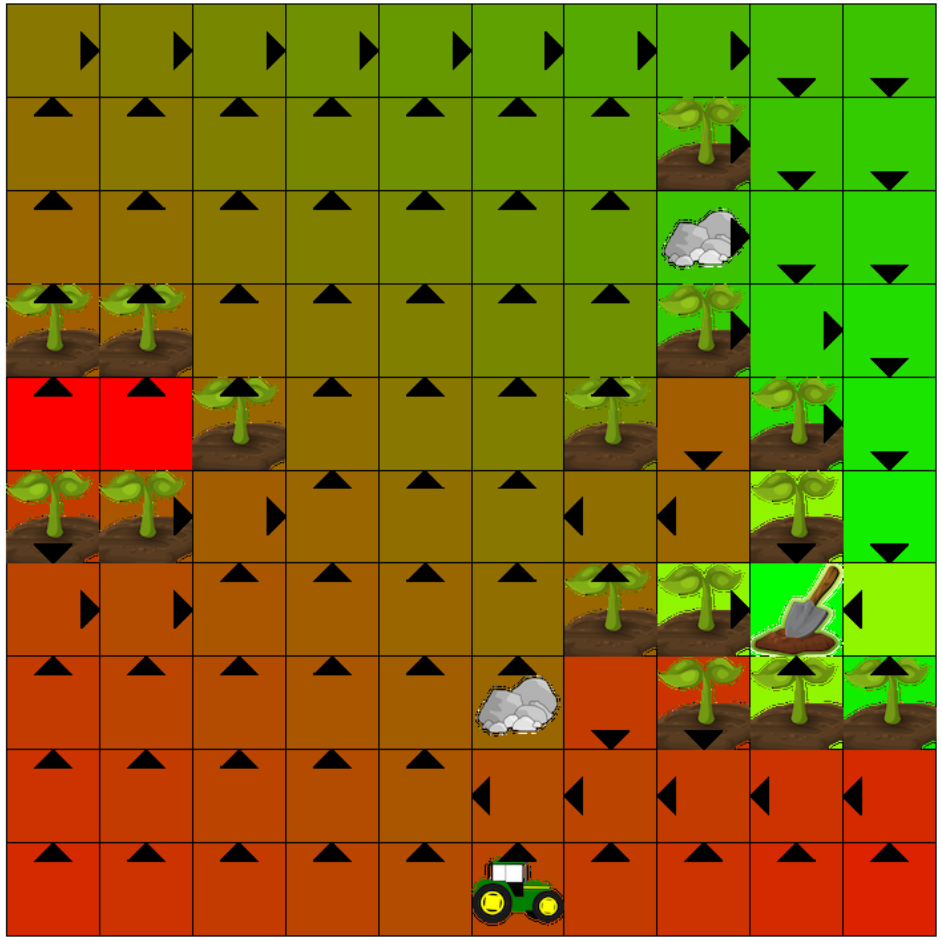


$v_\pi(s)$
π is uniform random for all states



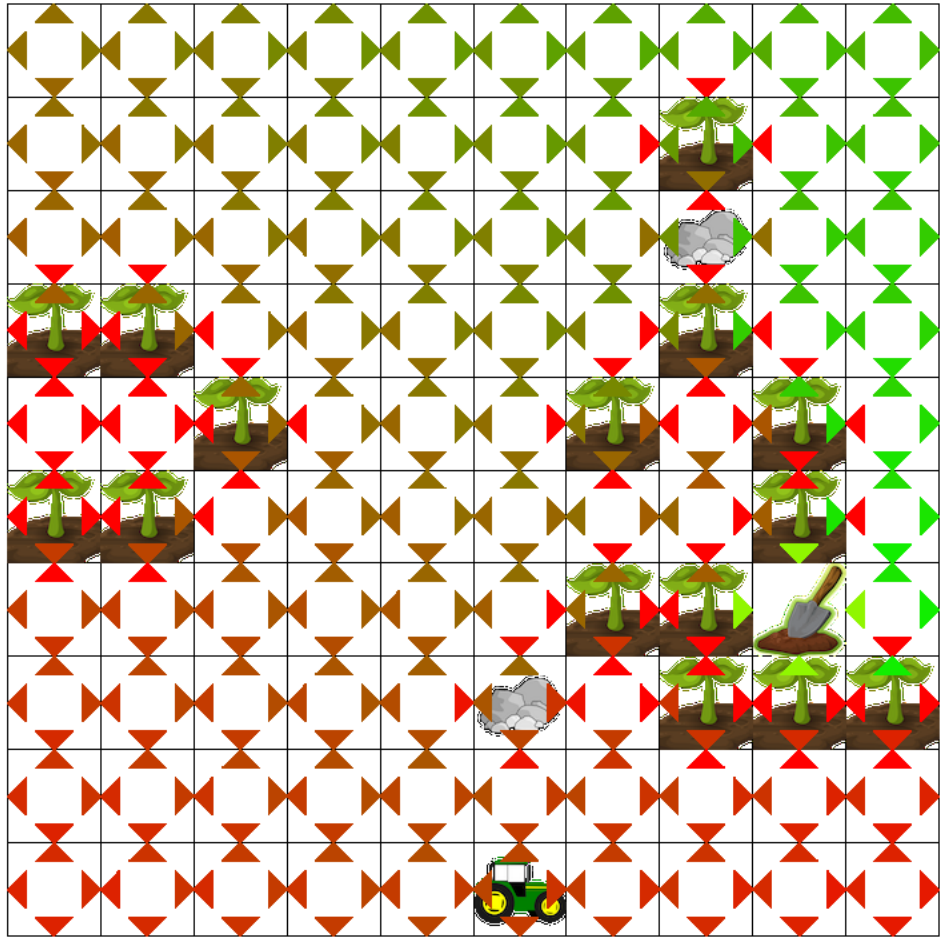
$v_\pi(s)$
π says to go up if below goal, go down if above goal, otherwise, go in the direction of the goal

Optimal Value Function Visualizations



$$v_*(s)$$

$$\pi_* = \underset{a}{\operatorname{argmax}}\left(r(s,a) + \gamma \sum_{s'} p(s'|s,a)\, v_*(s')\right)$$



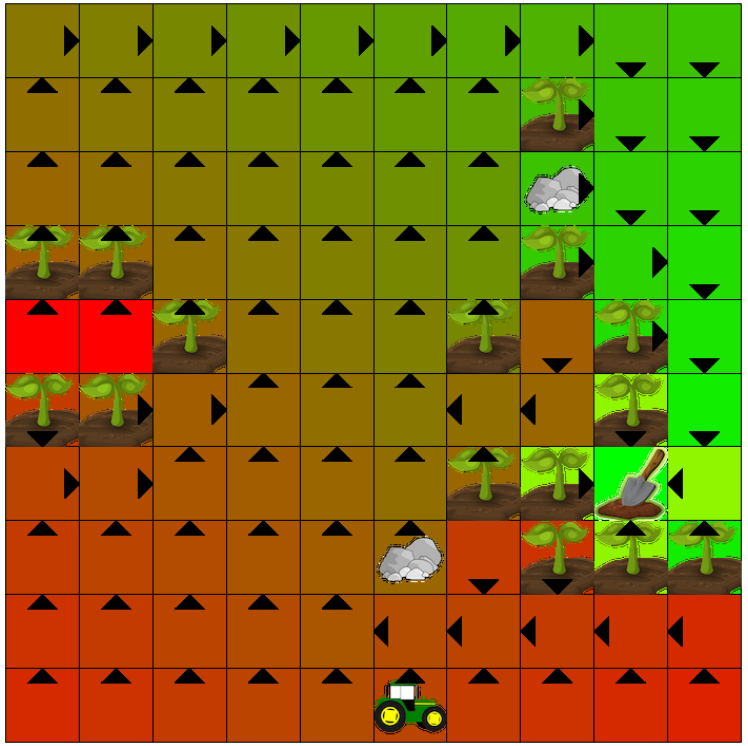
$$q_*(s,a)$$

$$\pi_* = \underset{a}{\operatorname{argmax}}\, q_*(s,a)$$

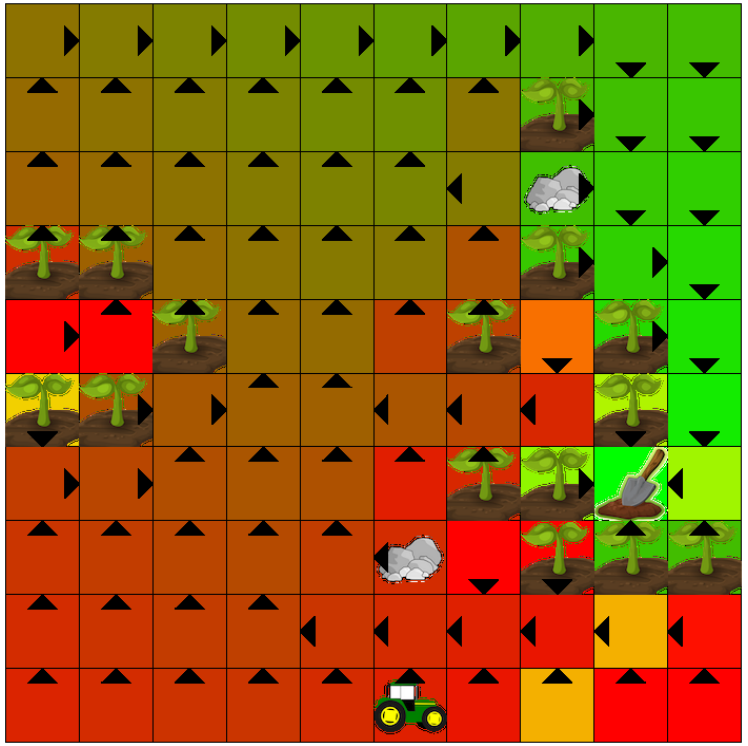
AI Farm: Strong Winds

- Wind blows you right with some probability p

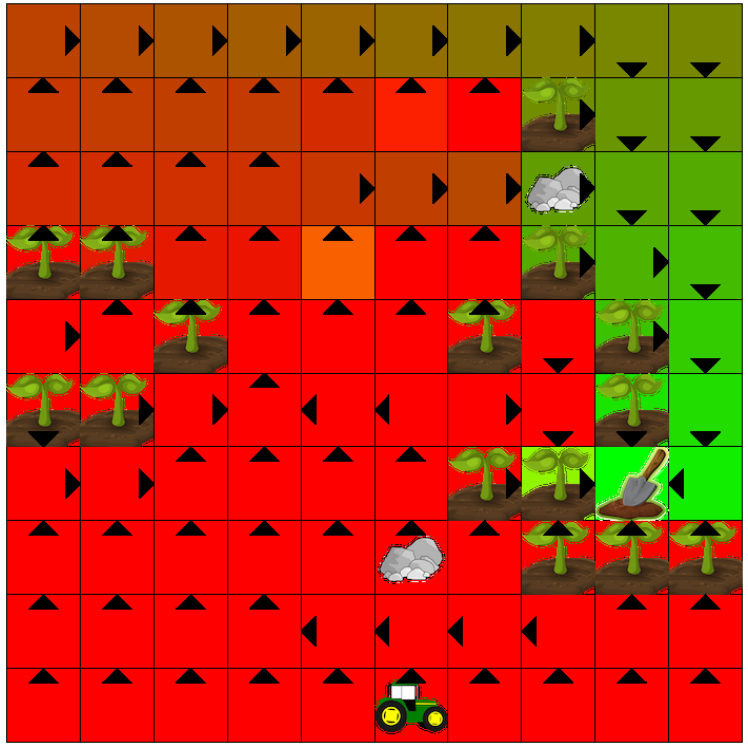
$p = 0$



$p = 0.1$



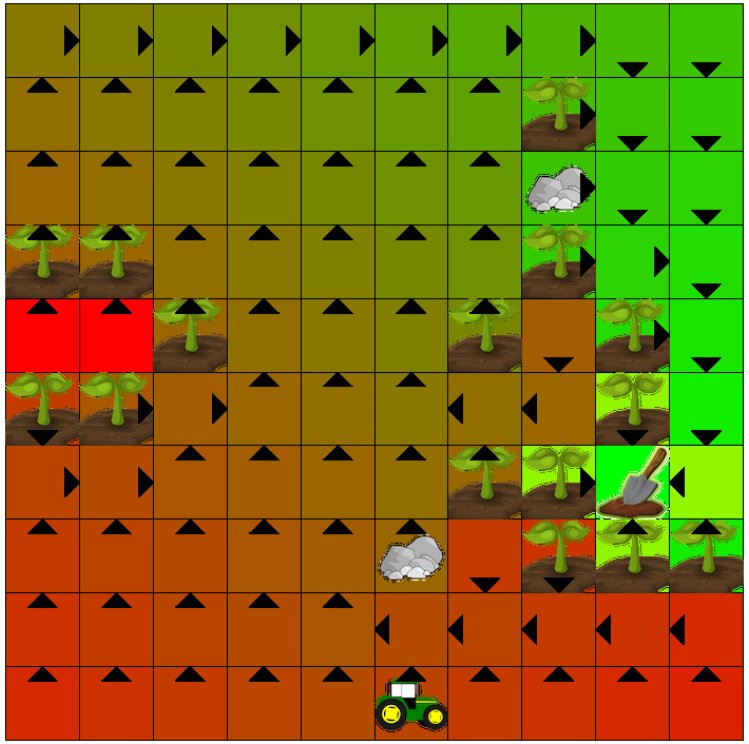
$p = 0.5$



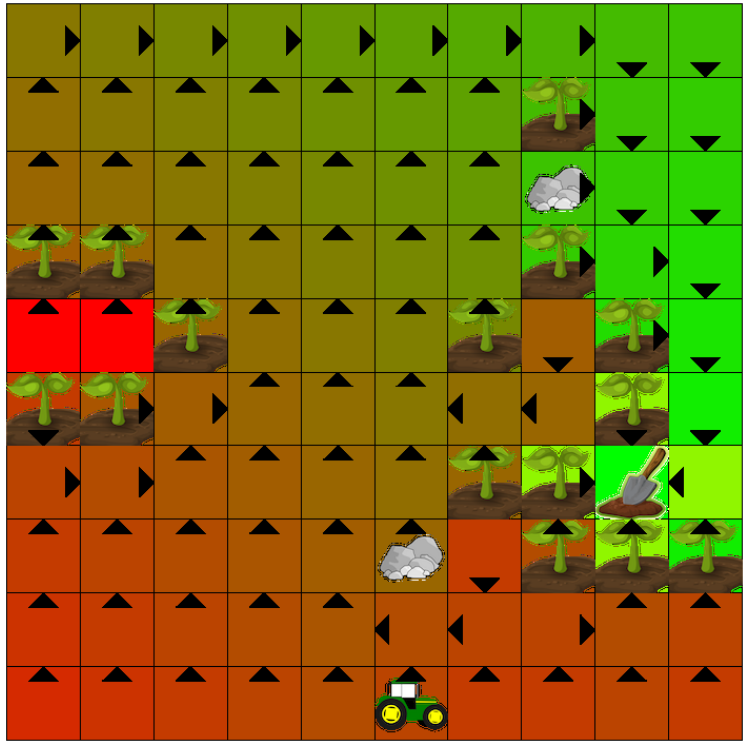
AI Farm: Effect of Rewards

- Reward r of driving on a plant

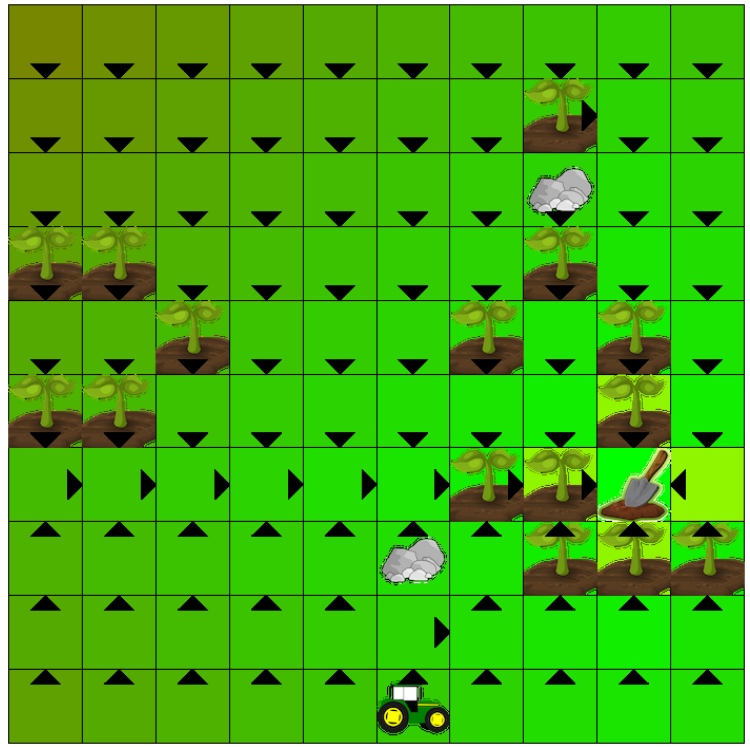
$r = -50$



$r = -20$



$r = -2$



Outline

- Markov Processes
- Markov Reward Processes
  - Solving MRPs
- Markov Decision Processes
  - Solving MDPs

Solving MDPs

- $v_\pi(s) = \mathbb{E}_{a\sim\pi(a|s)}[G_t|S_t = s] = \mathbb{E}_\pi[\sum_{k=0}^{\infty}\gamma^k R_{t+1+k}\,|S_t = s]$
- We can derive that
  - $v_\pi(s) = \sum_a \pi(a|s)(r(s,a) + \gamma \sum_{s'} p(s'|s,a)\, v_\pi(s'))$
  - Known as the Bellman equation
- We can then use dynamic programming methods to find v_*
- We can then behave greedily with respect to v_* to find a π_*
  - $\pi_* = \underset{a}{\text{argmax}}(r(s,a) + \gamma \sum_{s'} p(s'|s,a)\, v_*(s'))$

Bellman Equation

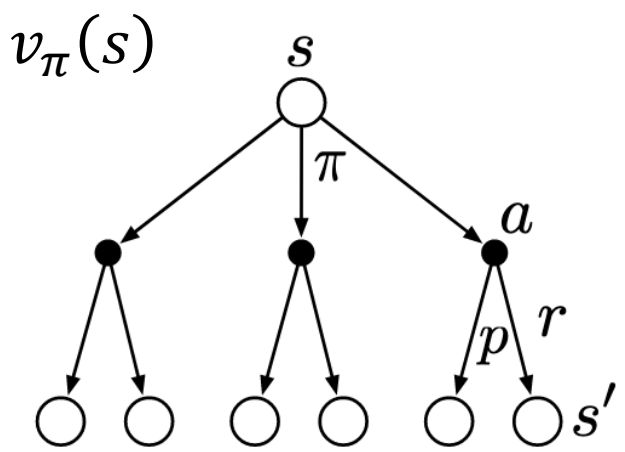
- $v_\pi(s) = \mathbb{E}_{a\sim\pi(a|s)}[G_t|S_t = s] = \mathbb{E}_\pi[\sum_{k=0}^{\infty}\gamma^k R_{t+1+k}\,|S_t = s]$
- $v_\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1}|S_t = s] = \mathbb{E}_\pi[R_{t+1}|S_t = s] + \gamma\mathbb{E}_\pi[G_{t+1}|S_t = s]$
- $\mathbb{E}_\pi[R_{t+1}|S_t = s] = \sum_r p(r|s)r = \sum_a\sum_r p(r,a|s)r = \sum_a\sum_r p(r|s,a)\pi(a|s)r$
- $\mathbb{E}_\pi[R_{t+1}|S_t = s] = \sum_a \pi(a|s)\sum_r p(r|s,a)r = \sum_a \pi(a|s)r(s,a)$
- $\mathbb{E}_\pi[G_{t+1}|S_t = s] = \sum_a \pi(a|s)\sum_{s'} p(s'|s,a)\mathbb{E}_\pi[G_{t+1}|S_{t+1} = s']$
- $v_\pi(s) = \sum_a \pi(a|s)r(s,a) + \gamma\sum_a \pi(a|s)\sum_{s'} p(s'|s,a)v_\pi(s')$
- $v_\pi(s) = \sum_a \pi(a|s)(r(s,a) + \gamma\sum_{s'} p(s'|s,a)\, v_\pi(s'))$
  - $p(s'|s,a) = \sum_{r\in\mathcal{R}} p(s',r|s,a)$
  - $r(s,a\,) = \sum_{r\in\mathcal{R}} r\sum_{s'\in\mathcal{S}} p(s',r|s,a) = \mathbb{E}[R_{t+1}|S_t = s, A_t = a]$

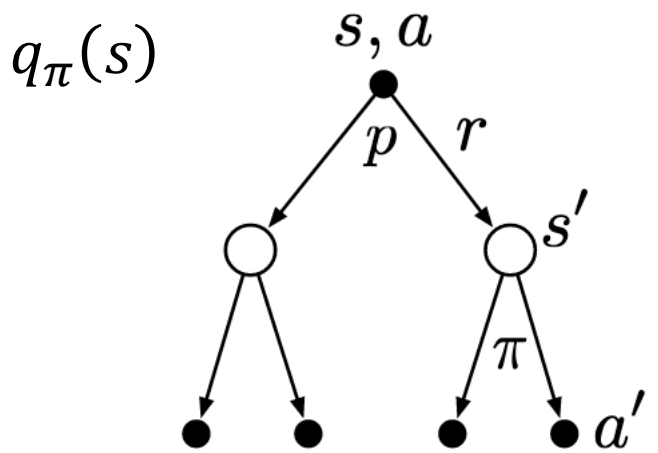
Bellman Equation

- $v_\pi(s) = \sum_a \pi(a|s) q_\pi(s,a)$
- $q_\pi(s,a) = r(s,a) + \gamma \sum_{s'} p(s'|s,a)\, v_\pi(s')$

Bellman Equations for v_π and q_π

- $v_\pi(s) = \sum_a \pi(a|s)(r(s,a) + \gamma \sum_{s'} p(s'|s,a)\, v_\pi(s'))$
- $q_\pi(s,a) = r(s,a) + \gamma \sum_{s'} p(s'|s,a) \sum_a \pi(a|s') q_\pi(s',a)$



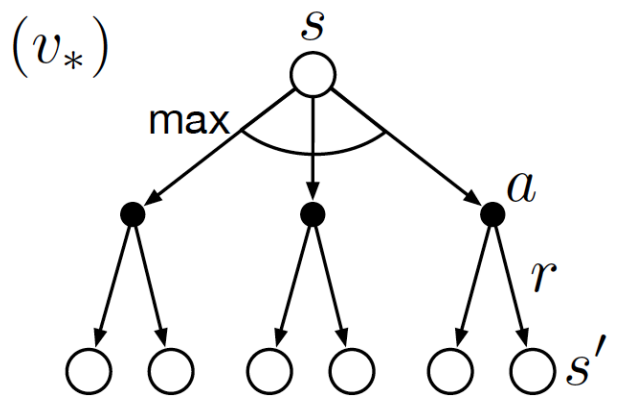


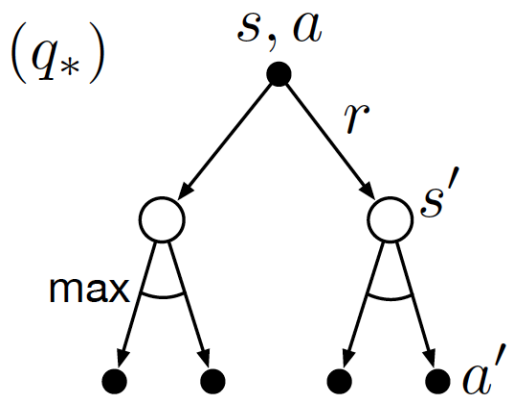
Bellman Optimality Equation

- $v_*(s) = \max_a q_*(s,a)$
- $q_*(s,a) = r(s,a) + \gamma \sum_{s'} p(s'|s,a)\, v_*(s')$

Bellman Optimality Equations for v_* and q_*

- $v_*(s) = \max_a (r(s,a) + \gamma \sum_{s'} p(s'|s,a)\, v_*(s'))$
- $q_*(s,a) = r(s,a) + \gamma \sum_{s'} p(s'|s,a) \max_{a'} q_*(s',a')$





Summary

- Reinforcement learning studies how we can maximize reward in sequential decision making problems
- Given a sequential decision making problem, we first must characterize our problem as a Markov decision process (MDPs)
  - The joint probability of the next state and reward is independent of the history given the current state and action
- The problem of computing the value of a given policy has optimal substructure (Bellman equation)
  - $v_\pi(s) = \sum_a \pi(a|s)(r(s,a) + \gamma \sum_{s'} p(s'|s,a)\, v_\pi(s'))$
- In the case of computing the value of an optimal policy (Bellman optimality equation)
  - $v_*(s) = \max_a (r(s,a) + \gamma \sum_{s'} p(s'|s,a)\, v_*(s'))$